

STM1 — Sensor Node Logic

By: Dhanush Raja A K

A Quick Note on Scope

Raw sensor data has no logic of its own. STM1's job is simply to **read** the sensors and send those values to STM2 over CAN. What STM1 *does* need is **validation logic** before sending, so it never transmits impossible or noisy readings. All calculations, fault decisions, and outputs (fan, buzzer, LCD) stay on STM2.

STM1 Flow

```
Read Temperature
|
Validate Temperature
|
Read Voltage
|
Validate Voltage
|
Read Current
|
Validate Current
|
Read Smoke
|
Validate Smoke
|
Pack CAN Frame
|
Send CAN Message
```

1. Temperature

Read

```
temperature = Read_Temperature();
```

Validate

```
if (temperature < -20 || temperature > 100)
    temp_sensor_fault = 1;
else
    temp_sensor_fault = 0;
```

Why these limits: the battery will never realistically hit -100°C or 250°C. Readings that extreme almost always mean the sensor is disconnected, the ADC glitched, or a wire is broken, not that the battery is actually at that temperature.

Sent over CAN:

```
Temperature    = 32.5 C
Sensor Status = OK
```

2. Voltage

Read

```
voltage = Read_Voltage();
```

Validate

```
if (voltage < 0 || voltage > 20)
    voltage_sensor_fault = 1;
else
    voltage_sensor_fault = 0;
```

Why these limits: the pack only ever operates between 12V and 16.8V. A reading like 50V means the sensor itself is faulty, not the battery.

Sent over CAN:

```
Voltage = 15.2 V
```

3. Current

Read

```
current = Read_Current();
```

Validate

```
if (current < -15 || current > 15)
    current_sensor_fault = 1;
else
    current_sensor_fault = 0;
```

Why these limits: the pack is rated at 10Ah, so current above 15A is well outside normal operation and almost certainly a sensor fault rather than real battery behavior.

Sent over CAN:

```
Current = 2.8 A
```

4. Smoke

Read

```
smoke = Read_ADC();
```

Validate

```
if (smoke < 0 || smoke > 4095)
    smoke_sensor_fault = 1;
else
    smoke_sensor_fault = 0;
```

Why these limits: the ADC is 12-bit, so its output is physically bounded to 0-4095. Anything outside that range is not a real reading.

Noise Filtering

Sensor readings fluctuate slightly from one sample to the next, so raw values shouldn't be sent as-is:

```
31, 32, 31, 33, 32, 31 ...
```

Sending every raw sample wastes bus bandwidth and produces a jittery signal on the receiving end. Instead, average several samples before sending:

```
Take 10 ADC samples
|
Average
|
Send
```

This gives a much more stable value on STM2's side.

Change Detection

STM1 doesn't need to send a CAN message if nothing meaningfully changed. For example:

- Previous voltage: 15.23V, current: 15.24V → difference is 0.01V, not worth sending.
- Previous voltage: 15.23V, current: 14.90V → real change, send immediately.

```
if (abs(voltage - last_voltage) > 0.05)
    SendCAN();
```

This significantly cuts down unnecessary CAN traffic.

Sensor Timeout Detection

If a sensor value stays frozen while everything else is changing, that's a sign it's stuck rather than reporting a genuinely stable reading. For example, if temperature holds at exactly 32.5°C for 30 seconds while current is actively changing, that's suspicious. In that case, STM1 should raise a `TEMP_SENSOR_STUCK` flag.

Sensor Range Summary

Sensor	Normal Range	Invalid (Sensor Fault)
Temperature	0 - 60 C (project)	< -20 C or > 100 C
Voltage	12 - 16.8 V	< 0 V or > 20 V
Current	-10 A to +10 A	< -15 A or > 15 A
Smoke (ADC)	0 - 4095	Outside ADC range

These invalid ranges flag **sensor faults**, not battery faults. A battery fault is a separate condition handled by STM2's logic.

CAN Packet Layout

The CAN frame carries the four sensor values plus a fault-flag byte:

```
Byte 0-1 -> Temperature
Byte 2-3 -> Voltage
Byte 4-5 -> Current
Byte 6    -> Smoke
Byte 7    -> Fault flags
            bit 0: Temperature fault
            bit 1: Voltage fault
            bit 2: Current fault
```

bit 3: Smoke fault

Packing diagnostic flags alongside the raw measurements is standard practice in many industrial CAN systems, it lets the receiving node know immediately whether a value can be trusted.

STM1 — Final Logic Summary

```
Read Sensors
|
Validate Sensor Values
|
Filter Noise
|
Prepare CAN Data
|
Send Every 100 ms
```

STM1's role stays simple: measure, validate, filter, and transmit. STM2 handles everything downstream, calculations, fault detection, fan and buzzer control, and the LCD. Keeping this separation clean is what makes the overall system modular and easy to maintain.